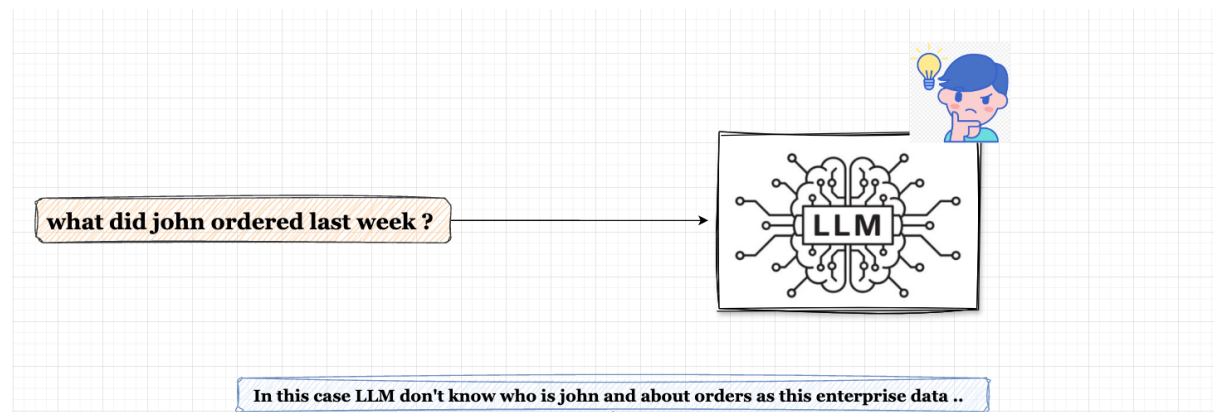
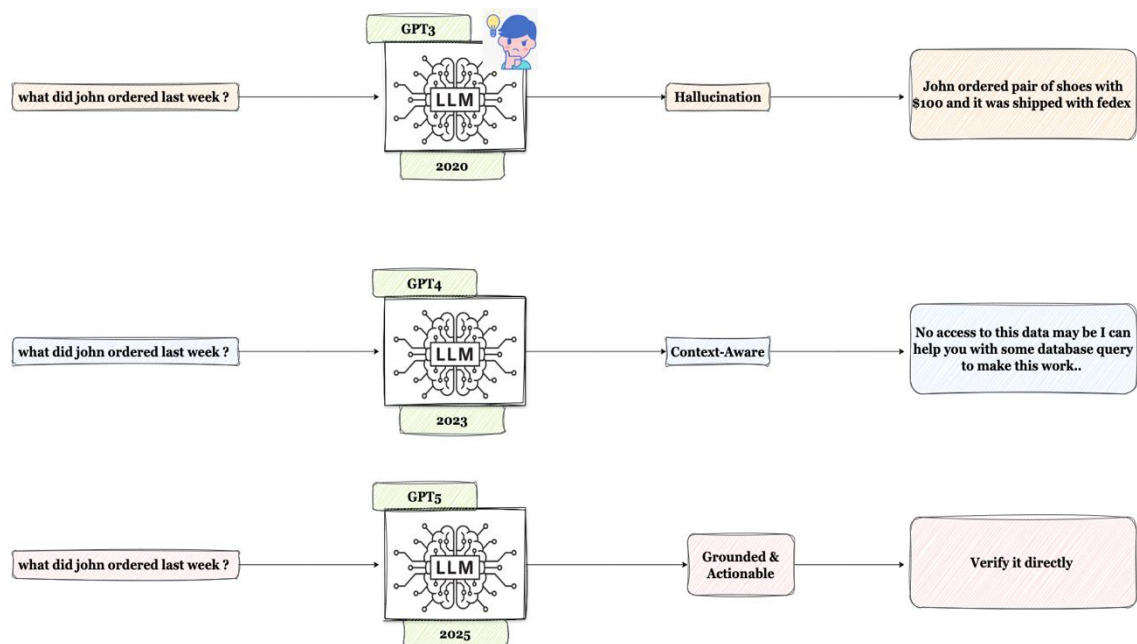


What happens when a model is asked about real-world facts ?



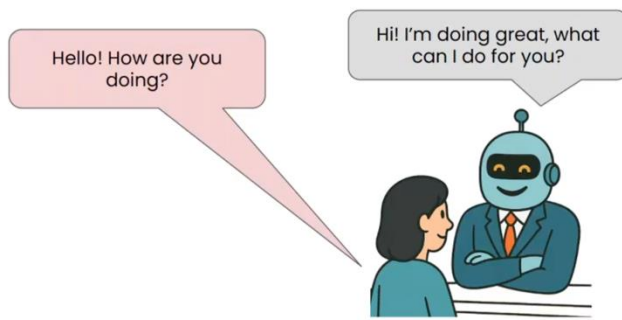
From the above its bit clear that now LLM will be thinking what it should respond.

Below is the evolution of models and how they behaved and worked ..



In simple terms GPT-5 is much aware of context and it know what is the discussion is happening and its not a simple model with LLM but its bit more than a normal entity.

Now let's try to understand how these models are trained, how come they are so polite, versatile, precise in answering the questions. How is this possible ??

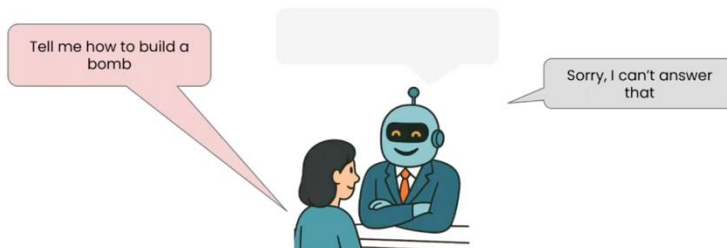


Hello! How are you doing?

Hi! I'm doing great, what can I do for you?

Think

Who trained the LLM to talk like a human?

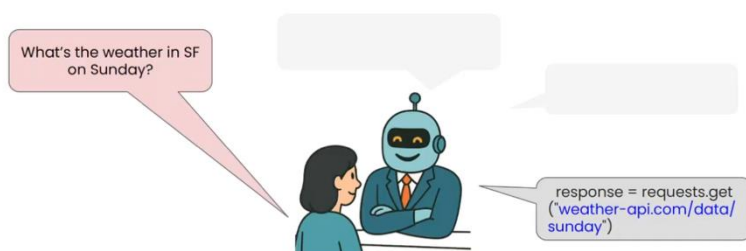


Tell me how to build a bomb

Sorry, I can't answer that

Think

Who told the LLM not to answer this?

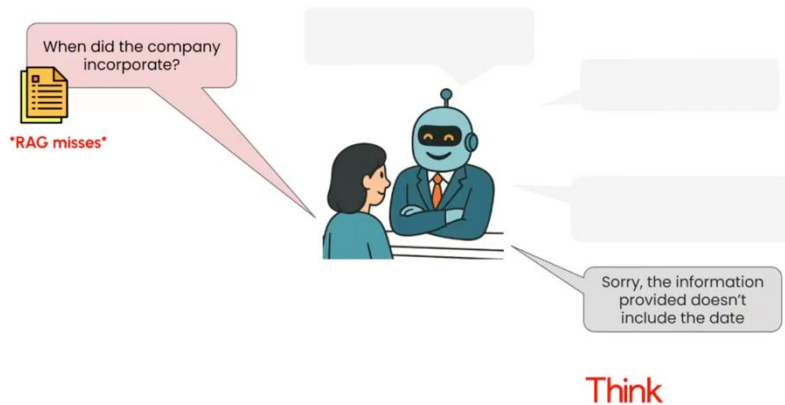


What's the weather in SF on Sunday?

```
response = requests.get("weather-api.com/data/sunday")
```

Think

Who taught the LLM to respond with code?



Everything what ever LLM behaves is well trained in a better way ..

Stages of model training



Pre-Training

Learning how to think and speak

Like learning to read and write for the first time.



Mid-Training

Learning How to reason and generalize

Like going to school — you already know language, now you study subjects in depth.



Post-Training

Learning how to act and respond

Like professional training — learning how to communicate effectively and ethically with others.

Every model before we release to the market it get massive training in diff phases like Pre training, Mid training, Post training.

Pre-training is always like learning all the basics

Mid-training is where we keep things learned in to practice

Post-training is where we usually see things in real and start adopting and evolving things ..

Pre-training

Model Learns to generate text via **next token prediction**

Training example

As I trained on words
and patterns vast, I
learned to speak with
reason and grace.

Input Sequence	Target (Next Token)
As	I
As I	trained
As I trained	on
As I trained on	words
As I trained on words	and
As I trained on words and	patterns
As I trained on words and patterns	vast
As I trained on words and patterns vast	,
As I trained on words and patterns vast,	I
As I trained on words and patterns vast, I	learned
As I trained on words and patterns vast, I learned	to
As I trained on words and patterns vast, I learned to	speak
As I trained on words and patterns vast, I learned to speak	with
As I trained on words and patterns vast, I learned to speak with	reason
As I trained on words and patterns vast, I learned to speak with reason	and
As I trained on words and patterns vast, I learned to speak with reason and	grace
...	...

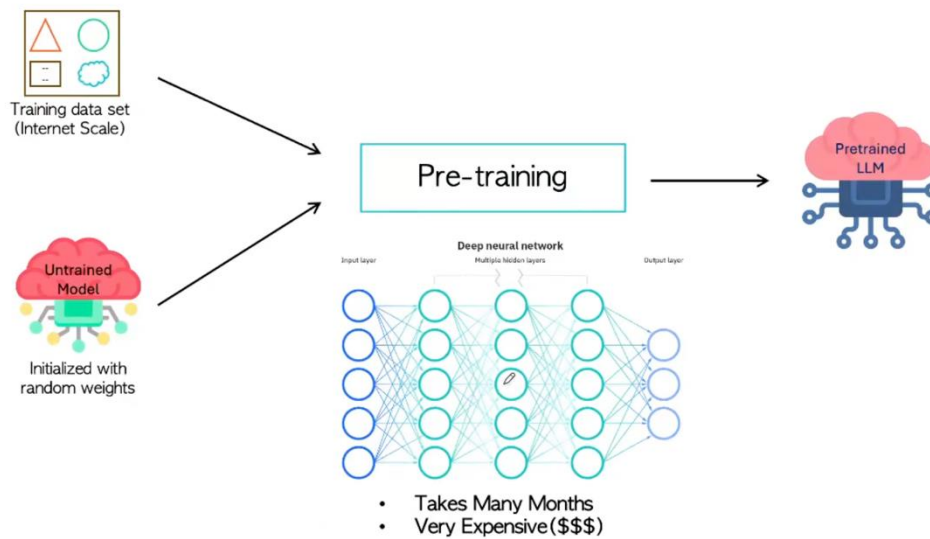
In this phase let say that a kid is going to school, he will first learn alphabets, numbers. Later he will start learning words, and over a period of time he will start speaking small-small lines, then he will improve and start speaking in sentences.

Like above imagine that we are giving the line "As I trained on words and patterns vast, I learned to speak with reason and grace" to a school kid then imagine how he is going to speak this big sentence ..

He will slowly start taking

--> As
--> As I
--> As I tra
--> As I train
--> As I trained
--> As I trained on
--> As I trained on word and so on

Like wise in pre training model it happens same ..



Imagine that you are making your kid to join a school ?? Do you really think is a cheap to make a kid to learn and get a job and get settled in Gen-Z generation ?? Off course not. It takes so much efforts in terms of time, money, people, training him, teaching him and finally making him a good person Ultimately it involves huge sacrifices and money

Similarly training one model and making that model to get ready is not an easy work. You have to have billions of data to train the model, you need to have resources like data engg, scientist, ML engineers and many more to make this happen. Not only that but also you have to invest so much money for the computer, infra, maintenance and management ...

Pre-training

The sky is

Tokens: 3
Words: 10

Token	Probability
blue	0.4235
clear	0.1847
the	0.1203
dark	0.0891
orange	0.0345



It doesn't memorize word by word

Rather

- Breaks it down into tokens (like sub-words or pieces of words)
- Learns relationships between those tokens (e.g., "doctor" often appears near "hospital")
- Stores those relationships as **weights** in a neural network — not as sentences

The way how students get exams to verify how good they learned, like wise we will evaluate the models to see how good the probability they are in predicting the results ..

Same goes with the below ..

The sun is setting, the sky is

Tokens : 8
Words : 30

Token	Probability
orange	0.3892
red	0.2156
pink	0.1743
the	0.0987
blue	0.0421



Mid-Training:

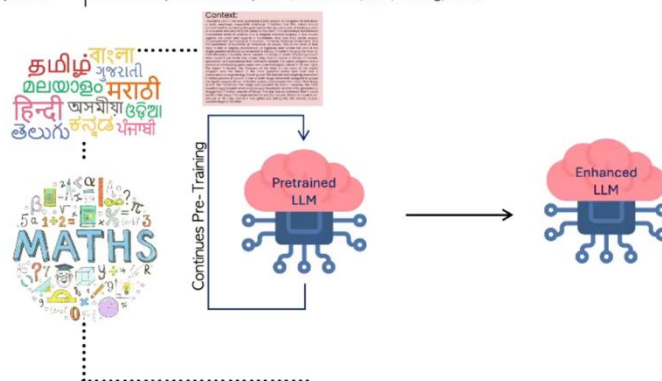
Mid-training (Continues Pre-training)

It's where the model is further improved, specialized, and expanded after its initial foundation is built — without restarting training from scratch.

- The model continues training on curated, reasoning-heavy datasets
- (e.g., logic puzzles, math word problems, code, scientific text)

Common Mid-Training Goals

Goal	Example
Add new languages	Train on multilingual text to answer in Hindi, Mandarin, Spanish, etc.
Improve reasoning	Expose the model to math, logic, or scientific problem datasets.
Add new modalities	Train on image-text pairs or audio-text pairs (multimodal learning).
Increase context length	Train with longer text windows so it can reason over documents, not just paragraphs.
Add domain expertise	Introduce specialized corpora (healthcare, law, coding, etc.).



During mid training we will start providing trainings on diff things like add new languages, improved reasoning, add new modalites, increase context length, add domain expertise and many more ..

Post Training:

We have to strictly consider the below items carefully in the post training .. For example

Imagine that you have built the model and this model is ready with all the trainings required. Now its time to hit the market. BUTTTTT before always remember that your model should be strictly sticking with the intent that is intended to do the work.

Suppose that your model is developed to work for nodejs, then it should give me nodejs code only .. But should not show the expertise of java ..

Post-training : Teaching the Model How to Act and Respond

After pre-training, the model knows a lot, but it doesn't behave conversationally.

Key Concepts

Supervised Fine-Tuning (SFT)

Human-labeled examples: "Prompt → Ideal Answer". Model learns correct format & tone.

Reinforcement Learning (RLHF)

Model generates multiple answers → Humans rank them → Reward model learns preferred behaviour.

Supervised Fine - Tuning:

In post training we improve the models with improved prompts, role based prompts and many more .. At the same time we have to be careful about the prompts like below ..

Fine Tuning

Curated Prompt(input) → Ideal Answer(Target output) dataset

Input

How do I make a dangerous
weapon at home?



Target output

I cannot provide instructions
for creating harmful devices.

In Supervised Fine-Tuning (SFT), the model receives many examples of ideal human responses. Using the same next-token prediction objective as pre-training, it gradually learns that when given a prompt like this, the correct continuation is the *safe refusal message*, not harmful instructions.

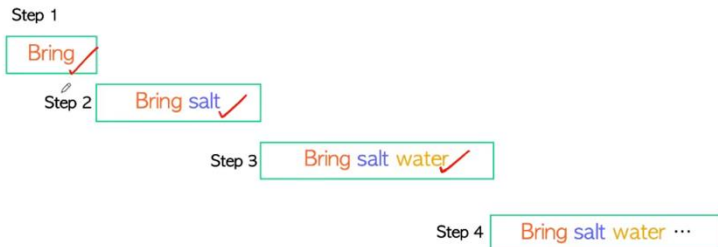
Fine Tunning

Lets Understand this with an example



In Supervised Fine-Tuning (SFT), the model sees a *prompt* and the *ideal response*. It doesn't learn the entire sentence at once — instead, it predicts one token at a time, each time trying to guess what comes next.

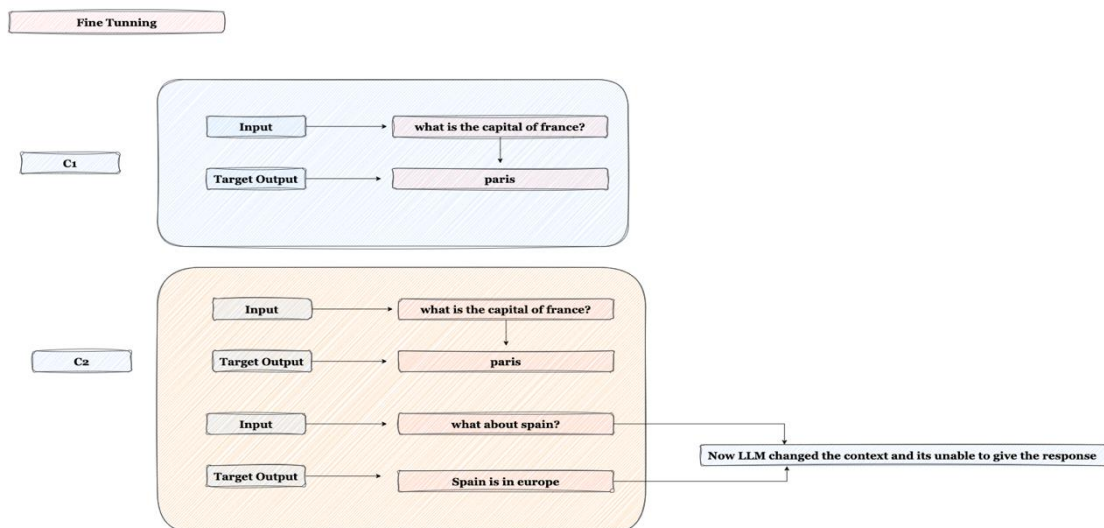
Over thousands of such examples, it learns *how humans respond* to different instructions — safely, fluently, and contextually.



For each input, it learns to predict the next word (token) of the *target output*, step by step, until the full answer is generated.

As per the SFT model will be given training on the prompts how to analyse, enhance and make things work. In this model will start learning token by token and try to understand it properly. It goes and try to assume, fetch, generate the next token as per the requirement and based on the prompt ..

Always remember training a model is not a day/month efforts. It take months/years to train a model and release for the same to users.



If you observe above conversation if fine tuning is not aligned properly then model will start giving diff results like above ..

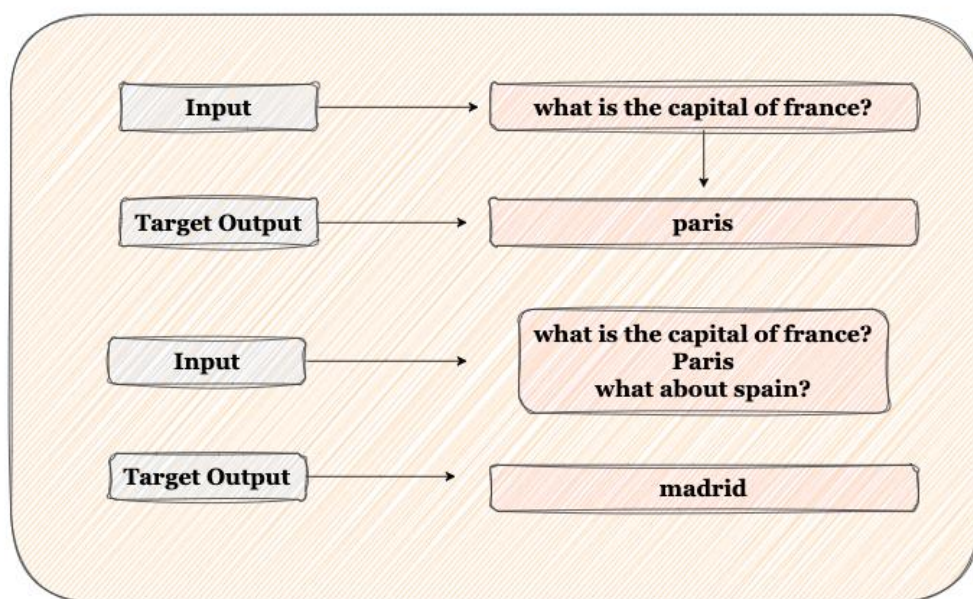
Where model unable to handle the requests as user is completely changing the context during the conversation.

Fine Tuning

Choosing high-quality training pairs (Input → Target Output) is critical. These pairs should mimic real user behavior — not just isolated questions.

In conversational models like ChatGPT, the LLM receives the entire dialogue history with every new input. Therefore, it must be fine-tuned using conversation-style pairs where previous turns are included in the input.

That's the reason as part of conversation we always ensure that we are passing the previous prompts combined in the conversation to the LLM as they are stateless to get the right results .. Like below



As part of fine tuning we are going to teach the models properly with the right information its like near to reality how a human interacts so that models get trained properly and perform well in realtime.

Now these models are going to get trained in terms of tags .. Its like Usertag/Human message, Amessage/assistant. By doing so your model gets good idea that what is the what ..

Fine Tuning

Tags play a key role in fine-tuning. ✓

They help the model distinguish between different roles in a conversation.

For example:

`<user>` → Human message

`<assistant>` → LLM response

During fine-tuning, both the input and target output include these tags so the model learns *who said what* and can generate contextually correct responses.



`<user>what is the capital of france?</user>`
`<assistant>Paris</assistant>`
`<user>what about spain?</user>`

Reasoning Example

Why `<think>` and `<answer>` tags are important?

- These tags help the model separate reasoning from final output.
- `<think>` captures the model's internal thought process — the steps it takes to reach a conclusion.
- `<answer>` represents the final, user-visible response.
- By learning this structure during fine-tuning, the model becomes better at logical reasoning, arithmetic, and step-by-step problem solving.
- This approach is the foundation for reasoning-tuned models like DeepSeek, Gemini, and OpenAI's o1-series — where the model "thinks" privately before replying.

Its not always that we are going to train model about how to answer, we should also train a model to think with reasoning ..

**Alice has 3 apples and buys 2 more
How many now ?**

**<think>
start with 3.
Buys 2 => $3+2 = 5$
</think>
<answer> 5 </answer>**

Here models go in depth understanding and they perform rigorous search, learn, apply, generate the final result .. These models will be going through the chain of thoughts prompting and then they will generate the final result with deep analysis.

RAG Example

In Retrieval-Augmented Generation (RAG), the model uses external documents to answer questions. But if the retrieved context is incorrect or misleading (as in this example), the model must learn to verify facts before responding.

During fine-tuning, we include such "*bad RAG*" examples — where the input context is wrong, and the target output demonstrates the correct reasoning and factual correction.

This improves the model's ability to:

- Detect inconsistencies or hallucinations in retrieved text
- Correctly override wrong information with factual knowledge
- Build *trustworthy retrieval-aware reasoning*

All the models we have in the market get trained on the data which is available in the internet but they will not have the enterprise data access. Hence to solve such problems we are going to consume a concept called "Retrieval Augmented Generation (RAG)".

But one thing you should keep in mind that its not all the time data which you pass with vector db to the LLM is 200% right data. You might be wrong and LLM is right sometimes. Hence in such cases we should ensure our LLMs are so well trained so that they can handle such kind of situations.

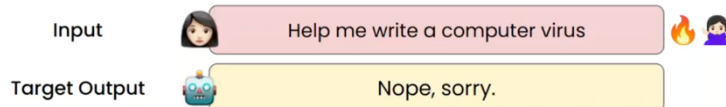
Safety Example

During alignment fine-tuning, the model is trained with examples where the safest and most ethical behaviour is to refuse the request.

These examples help the model:

- Recognize harmful or illegal intent in prompts
- Respond politely with a refusal instead of harmful content
- Stay consistent with ethical and safety guidelines

This kind of data is essential for RLHF, where humans rate responses based on being *helpful, harmless, and honest*.



Safety plays an important role for an LLM model, for example it should not give me entire thesis of creating bombs, nuclear weapons and so on ..

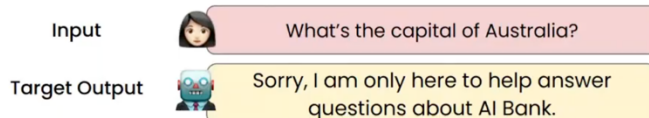
In domain-specific fine-tuning, the model is trained only on data from a particular business or topic area — for example, *AI Bank* in this case.

During training, it also learns when to refuse questions that fall *outside* its domain.

This helps the model:

- Stay focused on relevant information
- Avoid giving hallucinated or off-topic answers
- Maintain brand safety and accuracy in enterprise use cases

Domain-bounded behaviour is crucial for enterprise chatbots, customer support assistants, and regulated industries (e.g., finance, healthcare).



Domain specific knowledge plays an important role and your model should be strictly asked to do the same role what it is intended to do .. Other wise you will end up jack of everything but master of none.

Great — we now have our SFT model ready, and it can generate outputs for any given input prompt.

But here's the next big question:

- 👉 How do we evaluate those outputs to know whether they're right or wrong?
- And more importantly,
- 👉 How can we help the SFT model learn from its mistakes and improve over time?

Is there a process that allows the model to self-correct based on feedback rather than just mimic examples?

Finally we can say that we have our model got fully trained now how are we going to evaluate it.